

ARTI403: Image Processing

Lab 3: Image Manipulations using OpenCV Library

Student Name: Saad Abdulaziz Alshehri

Student ID: 2240006495

Objective

To explain how digital images are represented and manipulated in a computer, and to apply geometric transformations to images.

Tools

Python 3, Jupyter, OpenCV, NumPy, Matplotlib, scikit-image.

Procedural Steps

Step 1 (read an image): I used the astronaut image from scikit-image, saved it as input.jpg, and read it with OpenCV. The image size is 512×512 with 3 color channels.

```
img = cv2.imread('images/input.jpg')
```

Steps 2 to 4 (grayscale, save, and color spaces): I converted the image to grayscale, saved it as output.jpg, then converted it to YUV and separated the Y, U and V channels.

```
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
cv2.imwrite('images/output.jpg', gray_img)  
yuv_img = cv2.cvtColor(img, cv2.COLOR_BGR2YUV)
```

Input Image



Figure 1: Input image

Grayscale



Y channel



U channel



V channel



Figure 2: Grayscale and the Y, U, V channels

Assessment Task 1: Geometric Transformations

Increase the size

I enlarged the image 1.5 times using bicubic interpolation. The size changed from 512×512 to 768×768 .

```
resized = cv2.resize(img, None, fx=1.5, fy=1.5, interpolation=cv2.INTER_CUBIC)
```

Rotate by 120 degrees

I rotated the image by 120 degrees around its center. I made the output image bigger (699×699) so the corners are not cut.

```
M = cv2.getRotationMatrix2D(center, 120, 1.0)
rotated = cv2.warpAffine(img, M, (new_w, new_h))
```

Shear

I applied a shear on the x-axis with factor 0.3. The output width became 665.

```
M_shear = np.float32([[1, 0.3, 0], [0, 1, 0]])
sheared = cv2.warpAffine(img, M_shear, (new_w, h))
```

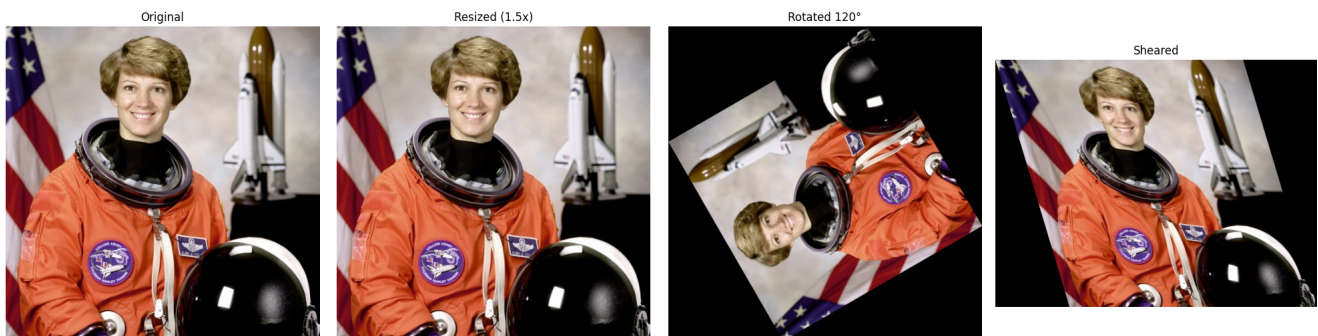


Figure 3: Original, resized, rotated and sheared images

Assessment Task 2: Intensity Transformations

I did these transformations on the grayscale image.

Negative image

```
negative = 255 - gray
```

Log transformation

I used $s = c \cdot \log(1 + r)$. I chose $c = 255 / \log(1 + \max) = 45.99$ so the output uses the full range from 0 to 255. The dark areas become brighter.

```
c = 255 / np.log(1 + float(np.max(gray)))
log_img = c * np.log(1 + gray.astype(np.float64))
```

Power-law (gamma)

I used $s = 255 \cdot (r / 255)^\gamma$ with $\gamma = 0.4$. A gamma smaller than 1 brightens the dark regions and increases the contrast there.

```
gamma_img = 255 * (gray / 255.0) ** 0.4
```



Figure 4: Grayscale, negative, log and power-law images